

A Private AI Assistant For The Lab

What we can build, how fast it will feel, and what it will cost

`libr-local-llm`

The idea, in one sentence

Put an AI assistant **on the lab's own computers**, so several people can use it at once — and so it is allowed to read patient data, which no outside service ever will be.

Today

You use Claude. It is excellent and fast. But every word you send leaves the building, so it can never touch a patient recording or transcript.

What this would add

An assistant living on our own machines. A bit slower. Not quite as clever. But it can look at the actual data, because nothing ever leaves.

This is **not** a replacement for Claude. It is a second tool that can go where Claude is not allowed to go.

First, the one idea everything else depends on

The model is not one brain. It is about 19,000 narrow specialists.

One knows about Python. One knows about Italian grammar. One knows about chemistry names. Nobody knows everything.

To write each word, the model picks the handful of specialists that word needs and **reads their notes**. Reading the notes is the work; everything else is quick. So the design question is: *where do we keep 19,000 sets of notes?*

instant
graphics card

only a few fit

fast
main memory

all 19,000 fit

very slow
shared storage

we avoid this

The reason this project is possible at all: our machines have enough main memory to hold *every one* of the 19,000 sets of notes. The assistant never has to go to slow storage while it is answering you.

Why it is slow: 600 specialists consulted, per word

Writing one word is not one lookup. The model works through the question in **75 rounds**, and in each round it consults **8** specialists.

**75 rounds $\times$ 8 specialists = 600 sets of notes read,
for every single word.**

That is the entire speed problem, and no clever engineering removes it. **Reading notes is the work.** We can keep the notes somewhere faster — that is what the previous slide was about — but we cannot read fewer of them.

And reading is only *half* the job. The other half is the thinking the model does once it has the notes in front of it. So even if reading were free, we would only be about **twice** as fast. Worth knowing before anyone promises ten times.

This is the *giant* model's problem

You still have to read **everybody's** notes.

Working together would only avoid slowing anyone down if you all wanted the **same** notes — and you never do.

Asking the giant model	your wait for a page
alone	about 1 minute
with 3 colleagues also asking	about 2–3 minutes
with 7 colleagues also asking	about 5 minutes

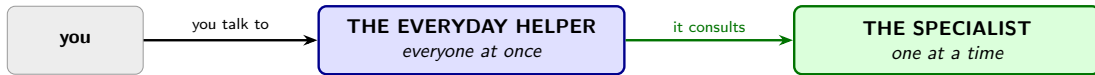
Measured, not estimated. The group does get through about **1.7 times** more work than taking turns — but that small gain is split eight ways while your own answer arrives **five times later**.

So the giant model is a one-at-a-time resource. It is not the thing a crowd should be talking to. **The next slide is why a smaller model escapes this entirely.**

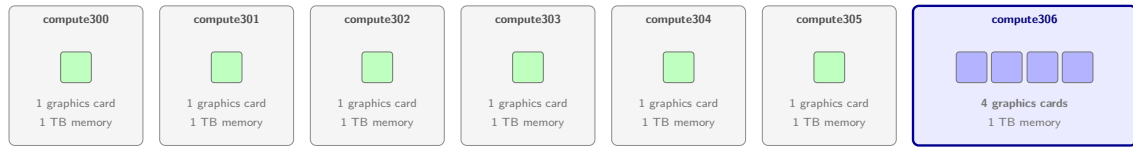
Why a smaller model escapes it — so: more than one helper

	the giant model	the everyday helper
notes kept	in ordinary memory	on the graphics cards
reading them	slow — this is the bottleneck	almost instant
so the slow part is	fetching notes	thinking
does sharing help?	no — everyone needs different notes	yes — one machine can do eight people's thinking at once far more cheaply than eight times over

That is the whole reason for building two. Fetching cannot be shared; thinking can. Make the model small enough that fetching stops being the problem, and a crowd suddenly costs almost nothing.



What we actually have to work with



7 computers. 10 graphics cards. 7 terabytes of memory.

The everyday helper needs *one* card. There are ten, so it is effectively never without one. A *bigger* version of it needs several cards together, which only compute306 can offer — so that one is a **bonus when compute306 is quiet**, never the thing the service depends on.

A warning about what more cards buy you. More cards means *more people served at once*. It does **not** make your own answer come back faster. Ten cards and one person is exactly as fast as one card and one person.

Helper 1 — the everyday one, and why it is always there

	the standard version	the bigger version
needs	one graphics card	several cards <i>together</i>
can run on	any of the 7 computers	compute306 only
how often available	essentially always	when compute306 is quiet
how good	good	better

You always get the best one that is free, and you never choose. Same command, same window. When the bigger one is up you get slightly better answers; when it is not, you get the standard one and nothing breaks.

This matters more than it sounds. While this was being written somebody else was using **one** of compute306's four cards — enough to stop a four-card-only design from starting at all. A helper that needs one card of ten does not have that problem.

Either version serves everybody at once. If the crowd grows we start more copies — which adds people, not speed.

Helper 2 — the specialist

How big	744 billion — the biggest thing we can run at all
Where it lives	one ordinary machine, using most of its memory
How many people	one at a time. Others wait in line
How fast	roughly a page of text per minute
What it is for	the hard question you would otherwise be stuck on

One catch worth planning around. Waking it up means reading 372 gigabytes off the shared storage. That takes about **27 minutes**. So we wake it in the morning and leave it awake, rather than starting it on demand.

It is slow because it is enormous, and it is worth having *because* it is enormous. A minute of waiting for a genuinely hard answer is a good trade. A minute of waiting to rename a variable is not — which is exactly why helper 1 exists.

How good are they, really — next to Claude

What you are asking for	standard helper	bigger helper	the specialist	Claude
Explain what this code does	close	close	close	yardstick
Write a small function to a spec	close	close	close	yardstick
Find a bug you can point at	worse	close	close	yardstick
Weigh two designs against each other	worse	worse	close	yardstick
Work through a written spec, many files	worse	worse	no too slow	yardstick
Stay on track over a long session	worse	worse	no	yardstick
Spot something subtle nobody flagged	no	worse	worse	yardstick
Know when it is wrong	no	no	no	imperfect too
<i>a page of text takes</i>	<i>20–30 s</i>	<i>20–30 s</i>	<i>about a minute</i>	<i>about 9 s</i>

close = you would rarely notice **worse** = noticeably, but usable **no** = use a different one

Read the shape, not the cells. The specialist is the *best* of ours at one hard question and the *worst* at a long piece of work — long work means many turns, and it is slow. **The fast helpers beat it on anything that needs iterating**, which is why it is consulted rather than talked to. And the last row is the honest one: **none of them reliably knows when it is wrong**, ours least of all. **This table is judgement, not measurement** — slide 16 is the test that replaces it

“Can I just write down what I want and let it code?”

Yes — with the everyday helper, and with limits worth knowing first.

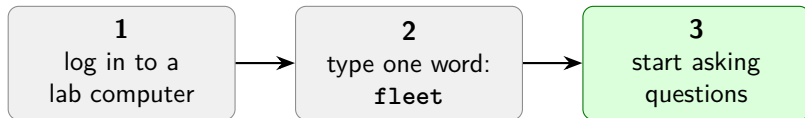
A moderate coding job, start to finish	How long	Sensible?
Claude, today	3–5 minutes	yes
the everyday helper	6–12 minutes	yes
the specialist	about 30 minutes	no — leave it alone

Two to four times slower than Claude for the same job. Write the description, start it, go and do something else for ten minutes. A real workflow, not a demo.

The catch is not about speed. To edit your files the model fills in a little form for each action, and because we run a *compressed* copy it **sometimes fills the form in wrong**. There is a built-in repair for the simple forms — open a file, run a command — but **not for the complicated ones**, and “change this line to that line” is a complicated one.

Start with small, clearly described jobs and check the result. How often it fills the form in wrong is one of the first things we measure — if it is often, “hand it a spec” is a demo, and we want to know that early.

What it looks like to use



That is the whole interface. One word. No settings, no flags, no remembering which helper to ask for. It starts anything that is not already running, waits for it, picks up your last conversation in that folder, and opens.

fleet	everything. This is the one you type
fleet status	is it running? is anyone waiting?
fleet down	switch it off now, rather than letting it idle off

You never ask for the specialist by name either. When a question is hard, the everyday helper goes and consults it, says so, and comes back with the answer — the same way you would ask a colleague to check something with an expert.

“Wait — do we need to pay for an API key?”

No. Nothing is bought, nothing is billed, and no word you type ever reaches Anthropic or anyone else outside the building.

There are two things called a “key” here, and only one is real.

The fake one

The Claude software refuses to start unless a certain box has *something* in it. So we type the word `local`. It is not an account, it is not checked by anyone, and it never leaves the machine. It is a seatbelt light, not a ticket.

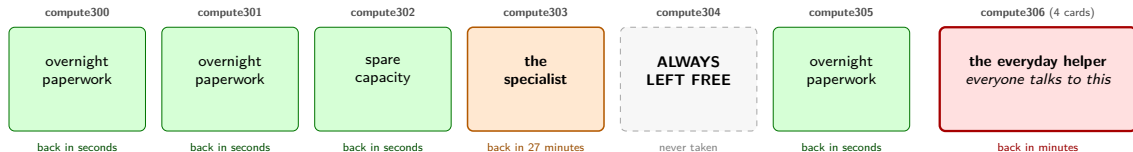
The real one

A password of our own, that we choose, protecting our own assistant. This one matters.

Why a password at all: these are **shared** computers. Anyone else logged in could otherwise walk up and use our assistant — spending our share of the lab's computing time, and asking it things we would be answerable for.

One specific hazard: conversations sit in numbered slots. If a stranger picks the number you are using, you do not get two conversations — you get one, tangled together.

How the machines get shared — and why nobody gets blocked



Of ten graphics cards, eight can be handed back within seconds, because what is running on them is paperwork that can be stopped mid-page and picked up later. One is never taken at all.

So how does a colleague get a machine? Two ways, and neither needs them to ask us. **One is already free** — we always leave one. And if they need more than that, the system sees their job waiting, finishes whatever answer is in progress, and hands a machine back on its own. Usually within a minute.

Being a good neighbour

Everything on the previous slide happens **by itself**. Nobody negotiates, nobody emails, nobody has to remember to switch anything off. **You do nothing.**

The subtle one is “waits”. Without it the system hands a computer back, notices a second later that it is short of computers, and grabs the same one straight back — beating the very colleague it was trying to help. It would look rude while running code written to be polite.

notices a colleague is stuck
finishes the answer you are waiting on first
hands the computer back, waits, takes it again later
switches off when unused, back on when needed
restarts itself after a crash
reconnects you when it moves to a different computer
holds your question while it is starting up

switch it off right now

automatic
automatic — it never cuts you off mid-sentence
automatic
automatic
automatic
automatic
automatic — you see “starting”, not an error

the only button you ever press

How we will know whether it is any good

Speed we can measure. **Whether it is *smart* enough is harder** — and we have an unusually good way to test it.

Use this project as the exam. Planning this system turned up three real discoveries about our own hardware that nobody had noticed. We know the answers now. Hand the local assistant the same starting point and see how many it finds on its own.

Would we bet on it?	The discovery it has to make
maybe probably probably not	one of our job queues can silently freeze the assistant mid-answer asking for memory quietly charges us for twice the computers why serving eight people at once cannot work

Grade this hardest: does it *hold its ground* when told it is wrong? One that folds and invents a better-sounding answer is worse than one that never spotted the problem.

What to expect: six people log in on Monday morning

Question	Honest answer
Can six people use it at once?	Yes. All six talk to the everyday helper, all at the same time, and it is built for exactly that.
How fast for each of them?	A page of text in 20–30 seconds. It does not noticeably slow down as colleagues join.
How does that compare to Claude?	Claude does that page in about 9 seconds . So: two to three times slower. Noticeable. Not painful.
What if two people need the specialist?	They queue . It serves one at a time, about a minute a page, so the second person waits about two minutes.
First thing Monday?	The specialist needs 27 minutes to wake up. We wake it before people arrive.
Will it be as clever as Claude?	For everyday work, close. For long, tangled problems across many files — no , and we should not pretend otherwise.

Everything above is an estimate built from real measurements on similar machines. The first thing we build is the test that replaces these numbers with ours.

The bottom line

**Six people. Their own terminal. A page of text every 20–30 seconds.
A genius down the hall for the hard ones.
And all of it allowed to read the data.**

What we are promising

- Everyone can use it at once
- Two to three times slower than Claude
- Good enough for everyday work
- Nothing ever leaves the building
- Nobody has to babysit it

What we are not promising

- It will not match Claude on hard problems
- The specialist serves one person at a time
- It uses a real share of the lab's computers
- Every number here still needs confirming

Slower. Not as clever. Allowed in the room where the data is.

That last one is the only reason to build it — and it is enough.